

2019-2020学年秋季学期

漏洞利用与攻防实践

*Exploiting Software Vulnerability-
Techniques and Practice*

报告：薄德芳 周怡

演示：王立岩 石鑫

漏洞利用与攻防实践

Exploiting Software Vulnerability-Techniques and Practice

[第十组] 模糊测试实践之变异 调度策略

2019.10.22

概要

- VUzzer: Application-aware Evolutionary Fuzzing
(NDSS'17)
- MOPT: Optimize Mutation Scheduling for Fuzzers
(USENIX'19)
- 实验：复现、改进MOPT

VUzzer: Application-aware Evolutionary Fuzzing

Sanjay Rawat*†, Vivek Jain‡, Ashish Kumar‡
, Lucian Cojocar*†, Cristiano Giuffrida*† and Herbert Bos*†
*Computer Science Institute, Vrije Universiteit Amsterdam

研究背景

- 基于Mutation的fuzzer一般都需要解决以下两个问题：
 - where to mutate ? (which offset to use for mutation)
 - what to mutate? (which value to use for mutation)
- AFL的解决方法是：随机
 - AFL选择、使用不同的operators来变异生成种子，where和what的答案由选择出的operator来决定
 - AFL保留产生new path的任何input，并对该input进行mutate之后作为新的输入，然后fuzz目标程序，并监控是否会执行new BB

研究背景

○ Magic Bytes

```
1 ...  
2 read(fd, buf, size);  
3 if (buf[5] == 0xD8 && buf[4] == 0xFF) // notice the order of CMPs  
4     ... some useful code ...  
5 else  
6     EXIT_ERROR("Invalid file\n");
```

若程序要执行到第4行，需要同时满足buf(5) == D8和buf(4) == FF，对于AFL随机变异出的input来说，非常困难的

研究背景

○ Deeper execution

```
15     if (buf[10] == '%' && buf[11] == '@') {
16         printf("2nd stop: on the way...\n");
17         if (strncmp(&buf[15], "MAZE", 4) == 0) // nested IF
18             ... some bug here ...
19     else {
20         printf("you just missed me...\n");
21         ... some other task ...
22         close(fd); return 0;
23     }
```

- 1) 假设某个种子满足第15行的条件，进入if分支；
- 2) 对于第17行的条件判断，无论条件是否真，程序会进入if分支或者else分支两者都发现一条新的路径；
- 3) 基于覆盖率的AFL 会把对应的input加入队列，参与下一次的种子选择

研究背景

○Markers

```
15  if (buf[10] == '%' && buf[11] == '@') {
16      printf("2nd stop: on the way...\n");
17      if (strncmp(&buf[15], "MAZE", 4) == 0) // nested IF
18          ... some bug here ...
19      else {
20          printf("you just missed me...\n");
21          ... some other task ...
22          close(fd); return 0;
23      }
```

- 1) 如果想要发现第18行的bugs，必须满足17行的条件；
- 2) 17行的判断语句，要求从buf[15]开始的4个字节是“MAZE”；
- 3) AFL的变异策略很难变异出合适的input

研究背景

○Nested Conditions

```
15  if (buf[10] == '%' && buf[11] == '@') {
16      printf("2nd stop: on the way...\n");
17      if (strncmp(&buf[15], "MAZE", 4) == 0) // nested IF
18          ... some bug here ...
19      else {
20          printf("you just missed me...\n");
21          ... some other task ...
22          close(fd); return 0;
23      }
24  } else {
25      ERROR("Invalid bytes");
26      ... some other task ...
27      close(fd); return 0;
28  }
```

- 1) 如果想要发现第18行的bugs，必须先满足15行的条件；
- 2) 对于AFL来说，进入第15行分支和进入第24行分支的input是同等重要的，进而有可能错过发现第18行代码的机会

论文概述

○研究问题

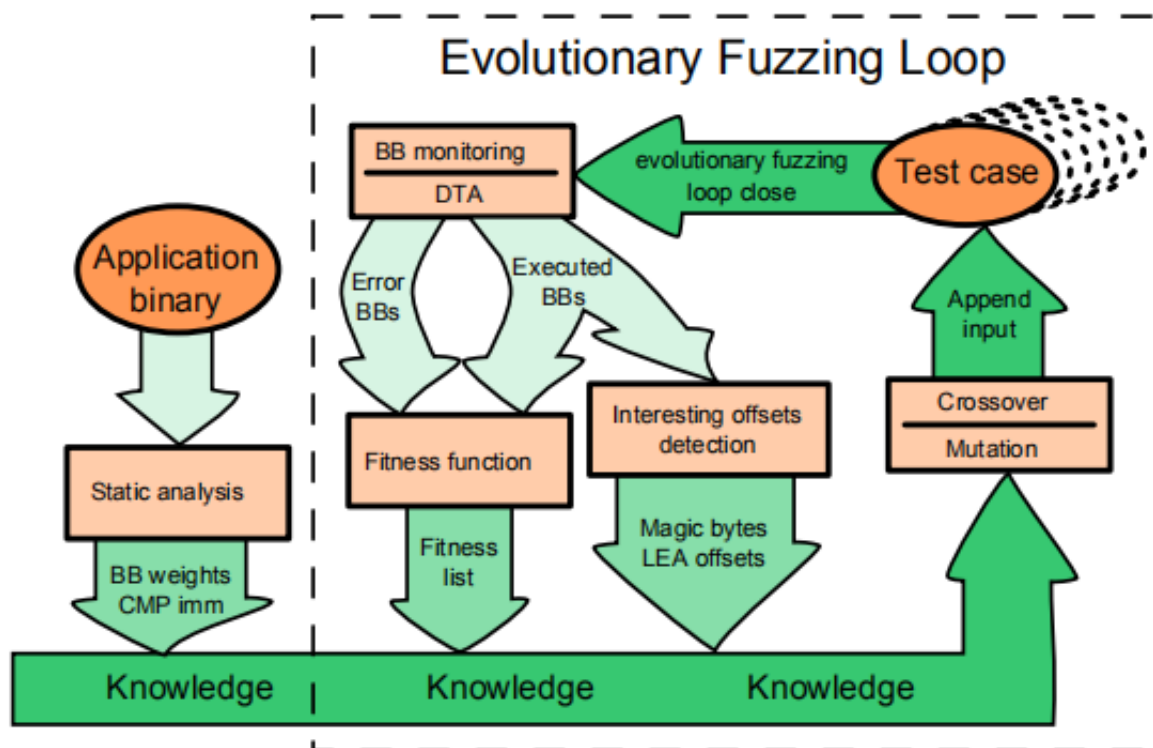
- 如AFL的fuzzers是application-agnostic，对不同应用程序的input格式采用相同变异策略，导致发现deep path的概率较低
- 现有的解决办法将符号执行技术与fuzzer相结合，缺点是削弱fuzzer的scalability（如Driller）

○解决方案

- Vuzzer——利用DTA（dynamic taint analysis）和 control/data flow分析，得到目标程序和有效input的特征，然后进行“smart”变异、种子选择，从而提高fuzzer的效率
 - 对目标程序分析：计算不同路径的优先级，在选择种子进行变异时优先考虑更深层次的路径，并降低对频繁出现的路径的优先级
 - 对有效input分析：解决where和what to mutate问题

论文实现

- Static analyzer : 在fuzzing之前运行，得到指令cmp和lea的相关信息（如操作数等）、计算BB的权重LBB；



论文实现

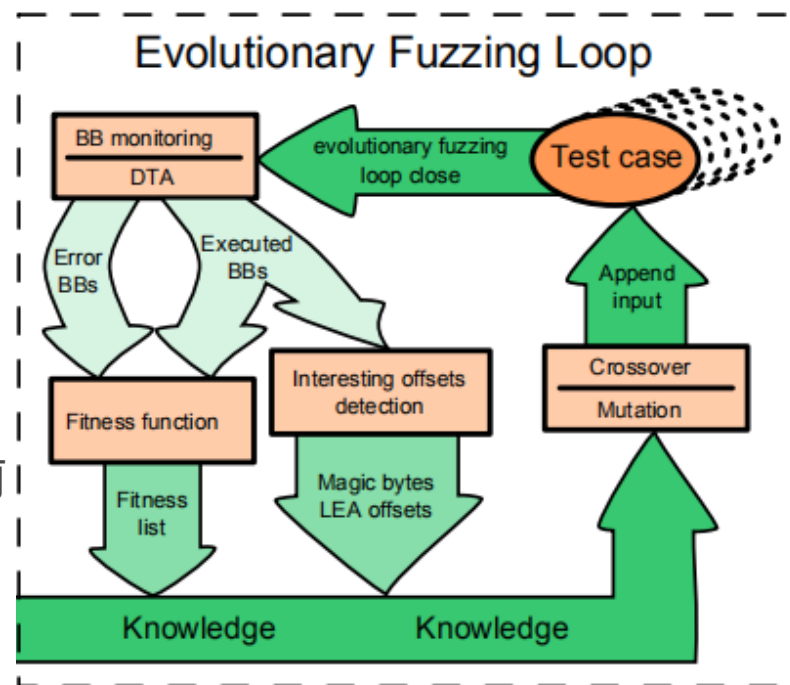
○ Valid input

- 利用DTA对提供的valid input进行分析，得到cmp和lea指令作用于input上offsets，即cmpi = (offset,value)和leai = (index)；（cmp指令中的value包含可能的magic-byte）
- 预运行：在fuzzing loop开始之前，以valid seeds作为输入，运行target program，得到初始化的control-flow和data-flow、valid seeds执行过的所有BB集合，为EHB的检测做准备；

论文实现

总体框架

- ①从种子池SI中取出input i开始fuzz，记录其执行过的BB(i)；
- ②若i执行了新的BB，则标记i、使用DTA通过程序的data-flow来分析i的结构特点；
- ③计算该input i的得分（Fitness），并记录；
- ④mutation：在SI、tainted input和Lfit的前n%随机选择input，进行crossover和“smart mutation”，得到new cases；继续循环直至发现crash或者满足终止条件。



实现细节

BB权重计算（非EHB）

- 假如基本块b有多条outgoing edges，将这些outgoing edges记为集合out(b)，即

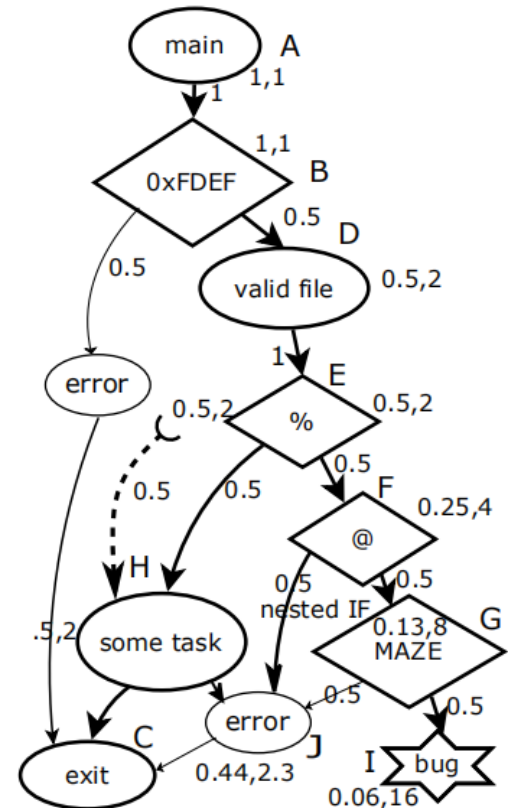
$$\forall e_{b \cdot} \in out(b), prob(e_{b \cdot}) = \frac{1}{|out(b)|}$$

b的执行概率表示为：

$$prob(b) = \sum_{c \in pred(b)} prob(c) * prob(e_{cb})$$

（其中pred(b)是基本块b的所有前驱的集合）

- 基本块b的权重：
$$w_b = \frac{1}{prob(b)}$$



实现细节

○ EHB (异常代码处理) 权重计算

○ 检测EHB :

- 初步分析：假设任一 $i \in SI$ ，有 $BB(i)$ 为 i 执行过的所有基本块，使 $Valid_{BB}$ 为有效输入 SI 执行过的 BB 的集合，假设 TR 为随机 input 的集合，则：

$$EHB = \{b : \forall k \in TR, b \in BB(k) \& b \notin Valid_{BB}\}$$

- 深度分析：假设 I 为一次 fuzz 产生的 input 的集合，有 $BB(I)$ 为所有 I 执行过的基本块，若基本块 b ($b \in BB(I)$) 和 I 中的 90% 有关系，且 b 不在 $Valid_{BB}$ 中，则 b 为 EHB。即：

$$EHB = \{b : \forall k \in P(I), s.t. |k| > |I| * 0.9, b \in BB(k) \& b \notin Valid_{BB}\}$$

实现细节

○ EHB（异常代码处理）权重计算

- 计算权重：由于EHB在所有BB中占比很小，故在计算input的fi（得分）时EHB的影响就相对较小；故引进参数 μ （ $0.1 \leq \mu \leq 1.0$ ）来解决此问题。对于输入i，则该输入i所执行过的EHB的权重为：

$$W_e = - \frac{|BB(i)| * \mu}{|EHB(i)|}$$

实现细节

○种子的 f_i （得分）计算

- 若当前input i 执行了一个新的非EHB，则该种子直接可参与到下一轮generation（该策略和AFL一致）
- 根据当前input i 执行过的BB来计算，有如下计算公式：

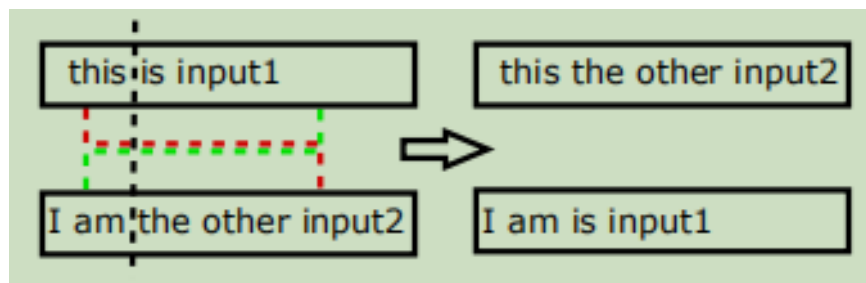
$$f_i = \begin{cases} \frac{\sum_{b \in \text{BB}(i)} \log(\text{Freq}(b)) W_b}{\log(l_i)} \text{BBNum} & \text{if } l_i > \text{LMAX} \\ \sum_{b \in \text{BB}(i)} \log(\text{Freq}(b)) W_b & \text{otherwise.} \end{cases}$$

其中 $\text{Freq}(b)$ 是基本块 b 被 i 执行过的次数， W_b 是基本块 b 的权重， l_i 是 i 的长度， LMAX 是input的长度限定。

实现细节

○ smart mutation (两个步骤)

○ crossover : 两个parents产生两个children



○ mutation : 一个parent产生一个child

- 首先在DTA分析的结果中随机选择offsets，并在这些offsets处插入程序中用于cmp指令的字节；
- 然后随机选择程序中用于lea指令的offsets，在这些offsets处替换上一些整数，如0、MAX_UNIT等；
- 对于所有cmp指令作用的位置，若 $op1 \neq op2$ ，则用 $op2$ 替换；
- 在Magic-byte相应的offsets处插入Magic-byte。

Mopt : Optimize Mutation Scheduling for Fuzzers

Chenyang Lv, ShouLing Ji, Chao Zhang
Zhejiang University
Tsinghua University

论文概述

○提出问题

- 基于变异的fuzzing产生new seeds的方法主要是通过变异；
- 现有的fuzzers大都以一定的distribution probability（概率分布）来选择mutation operator（变异操作），但效率不高。

○解决方法

- 提出Mopt算法，使Fuzzers以效率更高distribution probability来选择mutation operator变异生成种子；
- 将Mopt应用于基于变异的fuzzer之上，如AFL、AFLFast、以及VUzzer,fuzzing效率均有一定的提升。

研究背景

- 不同的Fuzzers会定义不同的mutation operators，以供变异阶段随机选择、使用。以AFL为例，AFL有11种mutation operators。

Table 1: Mutation operators defined by AFL [17].

Type	Meaning	Operators
bitflip	Invert one or several consecutive bits in a test case, where the stepover is 1 bit.	bitflip 1/1, bitflip 2/1, bitflip 4/1
byteflip	Invert one or several consecutive bytes in a test case, where the stepover is 8 bits.	bitflip 8/8, bitflip 16/8, bitflip 32/8
arithmetic inc/dec	Perform addition and subtraction operations on one byte or several consecutive bytes.	arith 8/8, arith 16/8, arith 32/8
interesting values	Replace bytes in the test cases with hard-coded interesting values.	interest 8/8, interest 16/8, interest 32/8
user extras	Overwrite or insert bytes in the test cases with user-provided tokens.	user (over), user (insert)
auto extras	Overwrite bytes in the test cases with tokens recognized by AFL during bitflip 1/1.	auto extras (over)
random bytes	Randomly select one byte of the test case and set the byte to a random value.	random byte
delete bytes	Randomly select several consecutive bytes and delete them.	delete bytes
insert bytes	Randomly copy some bytes from a test case and insert them to another location in this test case.	insert bytes
overwrite bytes	Randomly overwrite several consecutive bytes in a test case.	overwrite bytes
cross over	Splice two parts from two different test cases to form a new test case.	cross over

研究背景

- 不同的Fuzzers选择operators的策略不同。以AFL为例，AFL有三个变异阶段：deterministic stage、havoc stage和splicing stage；其中Havoc阶段利用一个的概率分布来选择operators。

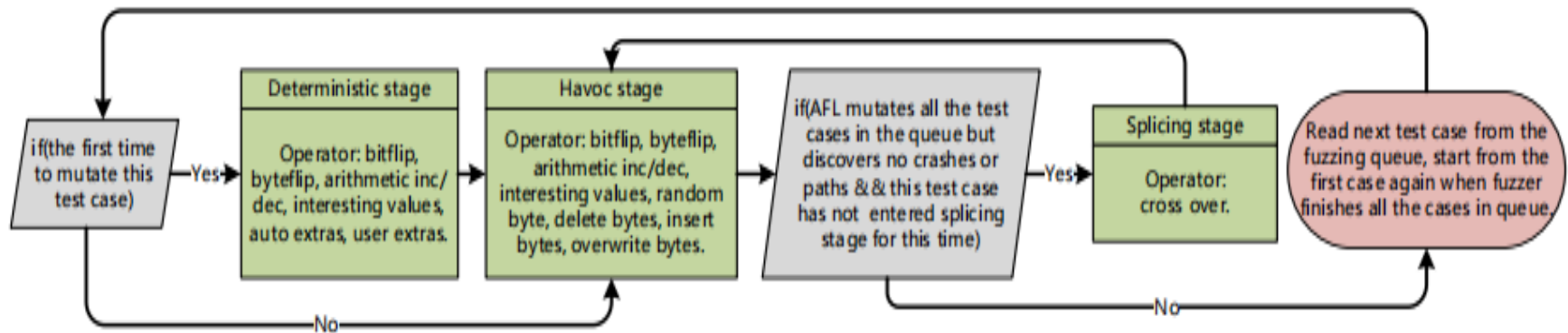


Figure 1: Three mutation scheduling schemes used in the three stages of AFL [17].

研究背景

- AFL中，在Deterministic阶段的schedule具有非常强烈的确定性的且耗时较长，而splicing阶段很少用到
- havoc阶段的调度策略更为通用，已被许多模糊测试者广泛采用。故本论文主要对havoc stage的distribution probability进行研究

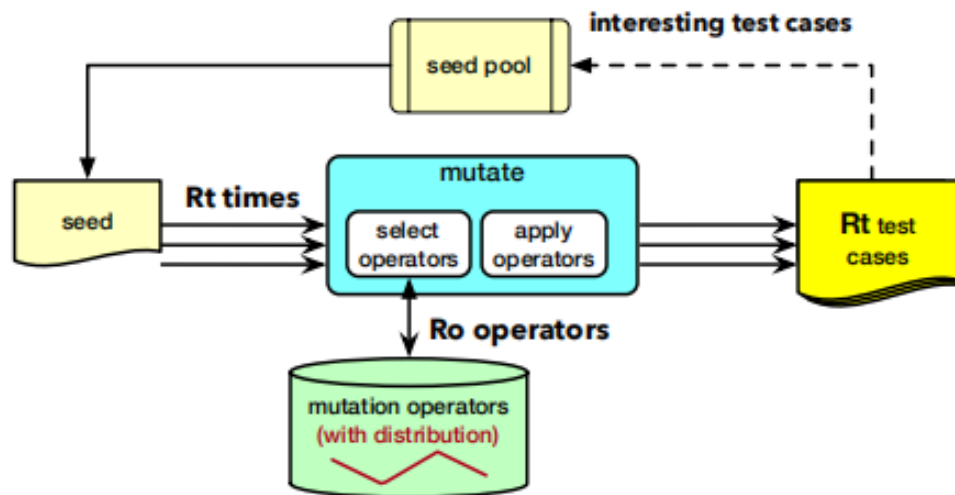


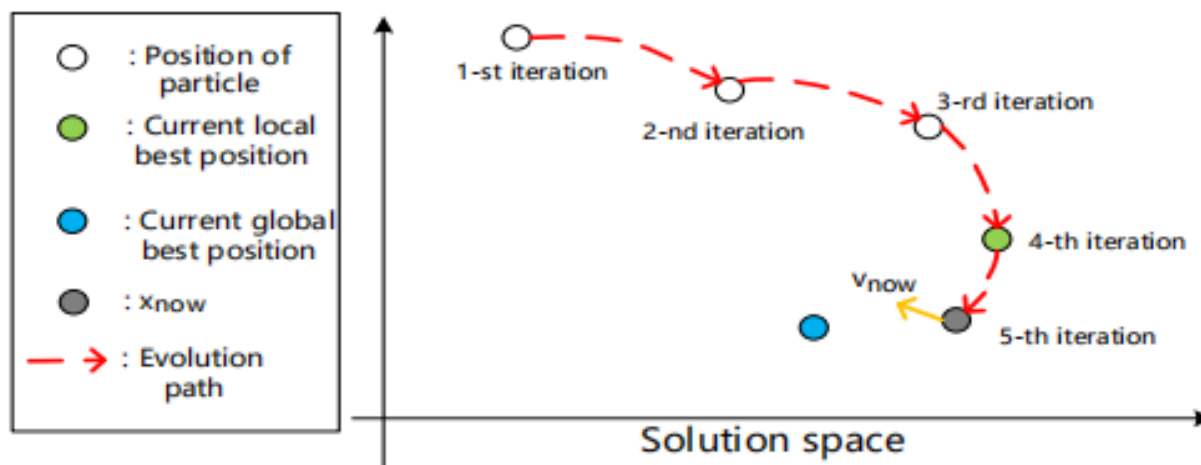
Figure 2: The general workflow of mutation-based fuzzing and mutation scheduling.

研究背景

- 不同的mutation operators在同一个target program上的效率是不同的；
- 同一个mutation operator在不同的target program上效率也是不同的；
- AFL在变异阶段花费大多数时间在deterministic stage，但是效率低；
- AFL在效率不高的operator上花费较多的时间，即已有的distribution probability的效率不高。

PSO算法

- PSO (Particle Swarm Optimization) 算法，使用多个 particles 搜索问题的最优解决方案。
- 每一轮迭代，更新particle的当前位置 X_{now} ；
- V_{now} 是当前的速度， L_{best} 是该particle历史达到的最优位置， G_{best} 是所有particles达到的最优位置



PSO算法

○更新算法如下：

$$v_{now}(P) \leftarrow w \times v_{now}(P) + r \times (L_{best}(P) - x_{now}(P)) \\ + r \times (G_{best} - x_{now}(P)).$$

$$x_{now}(P) \leftarrow x_{now}(P) + v_{now}(P).$$

w是惯性权重， $r \in (0, 1)$ 是随机位移权重

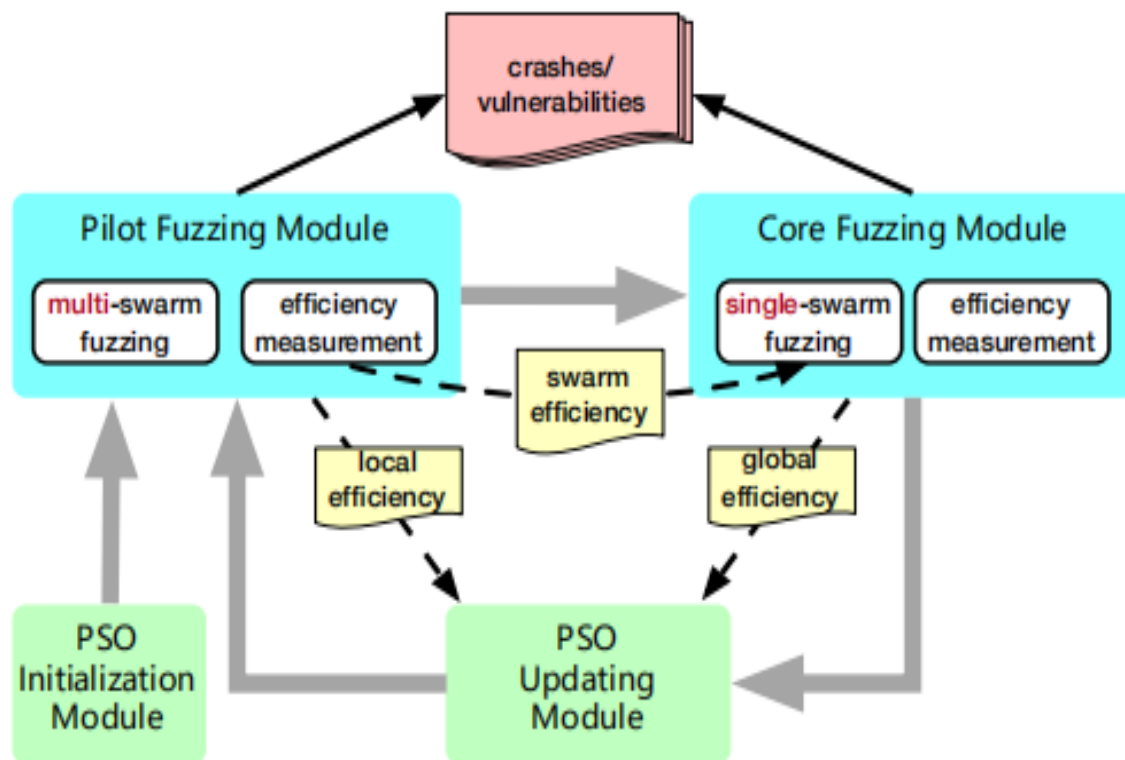
MOPT算法

- Mopt算法：搜寻最优的probability distribution；先搜索operator的最优选择概率，再构建最优probability distribution。
- 避免算法限于局部最优，采用多个swarm；
- X_{now} 表示某个operator被选择的概率；
- L_{best} 是某个operator在某个swarm中产生的interesting cases效率最高的概率；
- G_{best} 是某个operator在所有swarm中产生的interesting cases效率最高的概率；

MOPT算法

○整体框架

实现过程分为4个模块：PSO initialization、Pilot Fuzzing Module、Core Fuzzing Module和PSO Updating Module。



Pilot Fuzzing Module

- 同时运行多个swarms，计算每个operator的L_best和每个swarm的swarm_eff;
- 记录数据（对于每一个swarm）：
 - ①某个operator所产生的interesting cases的数量；
 - ②某个operator的调用次数；
 - ③该swarm产生的interesting cases的总数量；

$\text{local_eff}_i = \text{①} / \text{②}$, $\text{L_best} = \max \{\text{local_eff}_i, \dots\}$

$\text{swarm_eff} = \text{③} / \text{变异次数}$;

Core Fuzzing Module

- 以最优swarm(swarm_eff最大)的概率选择

operator进行变异；

- Select_Algorithm()算法如右；

- 记录：所有每个operator被选中的次数和对应产生的interesting cases数量。

```
double sele = ((double)(random()%10000)*0.0001);
//SAYF("select : %f\n",sele);
j_puppet = 0;
for (i_puppet = 0; i_puppet < operator_num; i_puppet++)
{
    if (unlikely(i_puppet == 0))
    {
        if (sele < probability_now[swarm_now][i_puppet])
            break;
    }
    else
    {
        if (sele < probability_now[swarm_now][i_puppet])
        {
            j_puppet = 1;
            break;
        }
    }
}
if (j_puppet == 1 && sele < probability_now[swarm_now][i_puppet-1])
    FATAL("error select_algorithm");
return i_puppet;
```

PSO Updating Module

- 根据Pilot阶段，计算L_best信息：
- Core Fuzzing提供的信息，计算G_best等信息：

$$G_best = \frac{operator_finds}{aloperators_finds}$$

- 更新operator的位置信息，更新公式：

$$\begin{aligned} v_{now}[S_i][P_j] &\leftarrow w \times v_{now}[S_i][P_j] \\ &\quad + r \times (L_{best}[S_i][P_j] - x_{now}[S_i][P_j]) \\ &\quad + r \times (G_{best}[P_j] - x_{now}[S_i][P_j]). \end{aligned}$$

$$x_{now}[S_i][P_j] \leftarrow x_{now}[S_i][P_j] + v_{now}[S_i][P_j].$$

Pacemaker Mode

- MOpt算法搜寻havoc stage最优的probability distribution ;
- AFL花费大多数的时间在deterministic stage ;
- Pacemaker mode实现在mutate one seed时，若在一定的限制时间（由用户设置时间限制）内未发现新的unique crash或path，则跳过deterministic stage。
- pacemaker有两种模式：-tmp和-ever
 - tmp模式：进入havoc stage后，若满足一定的条件，则返回deterministic stage；
 - ever模式：进入havoc stage后，不再返回deterministic stage。

实验部分

代码实现

Mopt 对于AFL的编译策略的选择方法进行了改进，为每个变异操作分配了不同的选取概率，采用粒子群最优算法选取最优的概率策略。

AFL主要函数：

- ◆ fuzzzone();

mopt主要函数：

- ◆ pilot_fuzzing();
- ◆ core_fuzzing();
- ◆ pso_updating();
- ◆ select_algorithm();

实验

- 分别使用AFL 和Mopt-AFL测试了5种目标程序：mp3gain-1.5.2, mpg321-0.3.2, objdump-2.30, jhead-3.20, exiv2-0.26,每个程序都跑了72小时左右。
- 测试环境：
 - Ubuntu 16.04 4核 (Virtual CPU) 16G *3
 - Ubuntu 16.04 1核 (Virtual CPU) 2G *1
- 参数设置：
 - mopt模式的参数采用：-L 1

结果：

Fuzzer + target	Unique crashes	Unique paths
Afl +mp3gain	122	993
Mopt+mp3gian	232(+90.2%)	2047(+106.1%)
Afl+mpg321	7	205
Mopt+mpg321	22(+214.3%)	255(+24.4%)
Afl + objdump	2	2324
Mopt+objdump	1(-50%)	2725(+17.3%)
Afl+jhead	2	592
Mopt+jhead	0(-100%)	623(+5%)
Afl+exiv2	0	2157
Mopt+exiv2	0 (+0%)	6875(+218.7%)

结果分析

- 由结果可以看出，相同时间内，mopt-afl比afl可以发现更多的crash和path，说明mopt的变异操作概率选择策略是有效的，这种方法对于不同的目标程序是自适应的，通过前期训练选择出最优的概率组合，并且不断更新，提高了afl的性能。
- 但是同时也可以看到，对于几个目标程序的效果并不是很好，发现的crash很少，原因在于：
 - 1，我们的实验时间比较短，文中为240小时
 - 2，某些程序本身bug比较少
 - 3，测试输入大部分为无效输入（useless testcase）。
 - 4，单次实验具有随机性，文中重复了30次。

简单改进

- 为了改进mopt，加快迭代更新效率，我们将pilot和core阶段的执行次数分别减小10倍，重新对jhead和mp3gain进行模糊测试，结果如下：

Afl+jhead	2	592
Mopt+jhead	0(-100%)	623(+5%)
Mopt2+jhead	0(-100%)	625(+5%)

Afl +mp3gain	122	993
Mopt+mp3gian	232(+90.2%)	2047(+106.1%)
Mopt2+mp3gian	238(+95.1%)	2156(+117.1%)

结果分析

- 改进之后可以看到，jhead的测试效果提高不大，只是多了两条路径，推测认为受限于jhead的输入文件都是无效输入，而模糊测试的种子输入十分重要。
- 改进之后的mopt对mp3gain的测试效果还是有提升的，在相同时间内，unique crash增加了6条，unique path增加了100多条，说明**适当**减少pilot和core阶段的执行次数对于**短时间的模糊测试**来说是有利的。

Q&A